



信息的表示和处理(1)

王晶

jwang@ruc.edu.cn, 信息楼124

2025年9月



本讲内容

- **Bit和Byte**
- **数字的机器表示**



Why Bit?

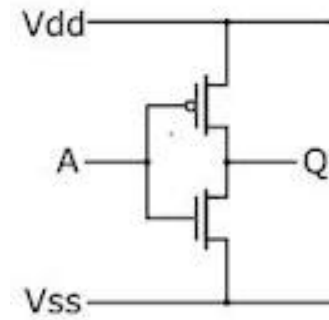
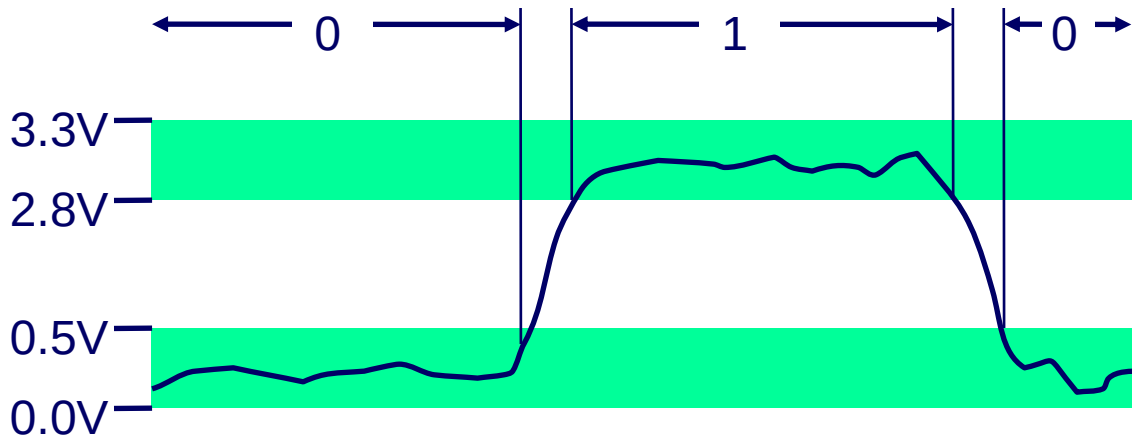
- 人类更习惯10进制 (Decimal)
 - Base-10, 10根手指
 - 已经使用超过1000年
 - 印度->阿拉伯->东西方





Why Bit?

- 模拟信号：连续，难存储，抗干扰能力差
- 数字信号：离散，易存储，便于逻辑运算
- 在计算机中表示信息方面，二进制比十进制更优秀
 - 对器件的要求低，
 - 简单、低成本、可靠
 - 提高电路密度



Binary is the most practical system to use!



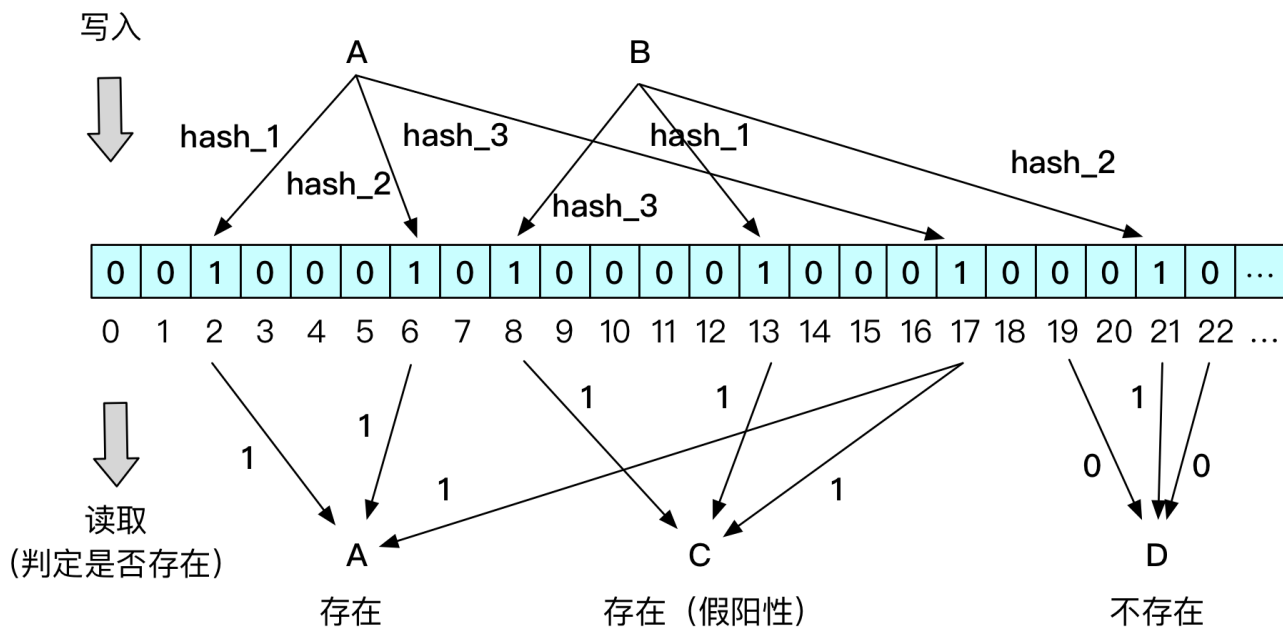
Why Bit?

- 计算机使用二进制表示信息
 - 二进制数字
 - 二进制值在构建机器时效果更好
 - 存储和处理信息
- 现代计算机存储和处理
 - 表示为双值信号的信息
 - 这些二进制数字是位
- 比特是数字革命的基础



Group Bits

- 单个的bit没太大用（bitmap, bloomfilter）
 - 因为alphabet太小（只有0和1两个符号）





Group Bits

- 英文还是有大量的词汇（符号组合）
 - 英文的alphabet包括26个符号，单个符号的表达力更强
- 因此，我们也可以用bits(而不是bit)来表示信息
 - 首先把bits分成组
 - 然后给可能的bit组合不同的解释，赋予其一定含义
- **8-bit组成a byte**
 - **Dr. Werner Buchholz in July 1956**
 - 在IBM Stretch计算设计的早期阶段

Why 8 bit?



维纳·布赫霍尔兹



Encoding Byte Values

- **Byte = 8 bits**
 - **Binary** 00000000_2 to 11111111_2
 - **Decimal:** 0_{10} to 255_{10}
 - **Hexadecimal** 00_{16} to FF_{16}
 - **Base 16 number representation**
 - **Use characters ‘0’ to ‘9’ and ‘A’ to ‘F’**
 - **Write $FA1D37B_{16}$ in C as**
 - **$0xFA1D37B$**
 - **$0xfa1d37b$**

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111



Group bits as

Numbers — Three encodings

- **Unsigned encoding**
 - 表示大于等于0的整数
 - 采用传统的二进制数字表达的规则
 - unsigned short, unsigned int, unsigned long
- **Two's-complement encoding**（二进制补码）
 - 可表示正、负整数（有符号数）
 - 是最常见的一种编码（主流计算机的默认整型编码）
 - short, int, long
- **Floating point encoding**
 - 近似表示实数，无法表示所有实数（很大/很小都不行）
 - 底为2的科学计数法方式
 - float, double



Value of Bits

二进制串的值:

Bits

01010

Value

$$0*2^4+1*2^3+0*2^2+1*2^1+0*2^0 = 10$$

世界上有10种人，
懂二进制的，
和不懂而二进制的！



Example Data Representations

C Data Type	Typical 32-bit	Typical 64-bit	x86-64
char	1	1	1
short	2	2	2
int	4	4	4
long	4	8	8
float	4	4	4
double	8	8	8
long double	–	–	10/16
pointer	4	8	8



'int' is not integer

- **Overflow (溢出)**
 - $200 * 300 * 400 * 500 = -884,901,888$
 - 乘积超过了整数的表示范围
- **满足交换律和结合律**
 - $(500 * 400) * (300 * 200)$
 - $((500 * 400) * 300) * 200$
 - $((200 * 500) * 300) * 400$
 - $400 * (200 * (300 * 500))$



'float' is not real number

- **Overflow and Underflow**
- **结合律可能不成立**
 - $(3.14+1e20)-1e20 = 0.0$
 - $3.14+(1e20-1e20) = 3.14$



进制转换

进制：进位计数制，逢X进一

二进制转十进制：

二进制数 $(N)_2$ 按权展开再相加，可计算得到该数的十进制表示。

$$\begin{aligned}(1101.0101)_2 &= (1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} \\ &\quad + 1 \cdot 2^{-2} + 0 \cdot 2^{-3} + 1 \cdot 2^{-4})_{10} \\ &= (8 + 4 + 0 + 1 + 0 + 0.25 + 0 + 0.0625)_{10} \\ &= (13.3125)_{10}\end{aligned}$$



进制转换

十进制转二进制

Value

102 (1100110)

Bits

$$102 = 51 * 2 + 0 \quad (0)$$

$$51 = 25 * 2 + 1 \quad (1)$$

$$25 = 12 * 2 + 1 \quad (1)$$

$$12 = 6 * 2 + 0 \quad (0)$$

$$6 = 3 * 2 + 0 \quad (0)$$

$$3 = 1 * 2 + 1 \quad (1)$$

$$1 = 0 * 2 + 1 \quad (1)$$



进制转换

十进制数转换成二进制数

- 整数部分和小数部分**分别转换**，各自得出结果后再合并。
- 对整数部分，采用**除2取余数法**。其规则如下：
 - 将十进制数除以2，所得余数（0或1）即为对应二进制数最低位的值。
 - 然后对上次所得商除以2，所得余数即为二进制数次低位的值，
 - 如此进行下去，直到商等于0为止，最后得的余数是所求二进制数最高位的值。
- 对小数部分，采用**乘2取整数法**。其规则如下：
 - 将十进制数乘以2，所得乘积的整数部分即为对应二进制小数最高位的值，
 - 然后对所余数的小数部分部分乘以2，所得乘积的整数部分为次高位的值，
 - 如此进行下去，直到乘积的小数部分为0，或结果已满足所需精度要求为止。



进制转换

十进制数转换成二进制数

例如：将 $(57.625)_{10}$ 转换成二进制。

• 整数部分的转换：

2	57		
2	28	余1	最低位
2	14	余0	
2	7	余0	
2	3	余1	
2	1	余1	
	0	余1	最高位

所以得出： $(57)_{10} = (111001)_2$



进制转换

• 小数部分的转换:

0.625		
× 2		
1.250	整数1	高位
0.250		
× 2		
0.500	整数0	
0.500		
× 2		
1.000	整数1	低位

↑

所以得出: $(0.625)_{10} = (0.101)_2$

总后得出: $(57.625)_{10} = (111001.101)_2$



十六进制Hexadecimal

- 十六进制的Alphabet中包含16个符号：‘0’ to ‘9’ and ‘A’ to ‘F’

- Write FA1D37B_{16} in C as

- **0x**FA1D37B or
- **0x**fa1d37b or
- FA1D37B**H** or
- fa1d37b**H**

- Byte = 8 bits

- Binary 00000000_2 to 11111111_2
- Decimal: 0_{10} to 255_{10}
- Hexadecimal 00_{16} to FF_{16}

Hex	Decimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111



Hexadecimal vs. Binary

0x173A4C

Hexadecimal	1	7	3	A	4	C
-------------	---	---	---	---	---	---

Binary	0001	0111	0011	1010	0100	1100
--------	------	------	------	------	------	------

1111001010110110110011

Binary	11	1100	1010	1101	1011	0011
--------	----	------	------	------	------	------

Hexadecimal	3	C	A	D	B	3
-------------	---	---	---	---	---	---

0x3CADB3



Hexadecimal vs. Decimal

Hexadecimal

0xA7

Decimal

$10 \times 16 + 7 = 167$

Decimal

$314156 = 19634 \times 16 + 12 \quad (\text{C})$

$19634 = 1227 \times 16 + 2 \quad (2)$

$1227 = 76 \times 16 + 11 \quad (\text{B})$

$76 = 4 \times 16 + 12 \quad (\text{C})$

$4 = 0 \times 16 + 4 \quad (4)$

Hexadecimal

0x4CB2C

十六进制和二进制转换练习：

① 0x39A7F8 -> [填空1]

② 1100 1001 0111 1011 -> [填空2]

③ 0xD5E4C -> [填空3]

④ 10 0110 1110 0111 1011 0101 -> [填空4]



Hexadecimal vs. Binary

- **0x39A7F8 -> 0011 1001 1010 0111 1111 000**
- **1100 1001 0111 1011 -> C97B**
- **0xD5E4C -> 1101 0101 1110 0100 1100**
- **10 0110 1110 0111 1011 0101 -> 2 6 E 7 B 5**



Decimal	Binary	Hexadecimal
167		
62		
188		
	0011 0111	
	1000 1000	
	1111 0011	
		0x52
		0xAC
		0xE7



Decimal, Hexadecimal, Binary

Decimal	Binary	Hexadecimal
167	1010 0111	0xA7
62	0011 1110	0x3E
188	1011 1100	0xBC
$3*16+7=55$	0011 0111	0x37
$8*16+8=136$	1000 1000	0x88
$15*16+3=243$	1111 0011	0xF3
$5*16+2=82$	0101 0010	0x52
$10*16+12=172$	1010 1100	0xAC
$14*16+7=231$	1110 0111	0xE7



Octal

八进制

- 八进制数 $(N)_8$ 按权展开再相加，可计算得到该数的十进制表示。

- 例如：

$$\begin{aligned}(15.24)_8 &= (1 \cdot 8^1 + 5 \cdot 8^0 + 2 \cdot 8^{-1} + 4 \cdot 8^{-2})_{10} \\ &= (8 + 5 + 0.25 + 0.0625)_{10} \\ &= (13.3125)_{10}\end{aligned}$$



Octal

二进制数与八进制数之间的转换

- 因为 $2^3=8$ ，所以3位二进制数与1位八进制数有直接对应关系，即3位二进制数可以直接写为1位八进制数，1位八进制数也可以直接写为3位二进制数。
- 将二进制数转换为八进制数的方法是：有整数又有小数情况，以小数点为界，整数部分自右至左每3位分一组，最后不足3位时左边用0补足；小数部分自左至右每3位分一组，最后不足3位时右边用0补足。
- 将八进制数转换为二进制数的方法：将八进制数的每一位用等值的3位二进制数代替。

$$\text{例: } (1101.0101)_2 = (001\ 101.\ 010\ 100)_2 = (15.\ 24)_8$$

$$\text{例: } (47.3)_8 = (100111.011)_2$$

进制转换（二进制小数位最多保留4位）

$$(157)_{10} = [\text{填空1}]_2$$

$$(102.625)_{10} = [\text{填空2}]_2$$

$$(36.15)_8 = [\text{填空3}]_{16}$$



课前练习

进制转换（二进制小数位最多保留4位）

$$(102.625)_{10} = \text{[填空2]}_2$$

十六进制和二进制转换练习：

0x39A7F8 -> [填空1]

1100 1001 0111 1011 -> [填空1]



小白鼠验毒问题之求解

测试一下你的思维...

有**1000**瓶水，其中有一瓶有毒，小白鼠喝了带毒的水**24**小时内就会死亡，
问：至少要多少只小白鼠才能在**24**小时内鉴别出哪瓶水有毒？



.....

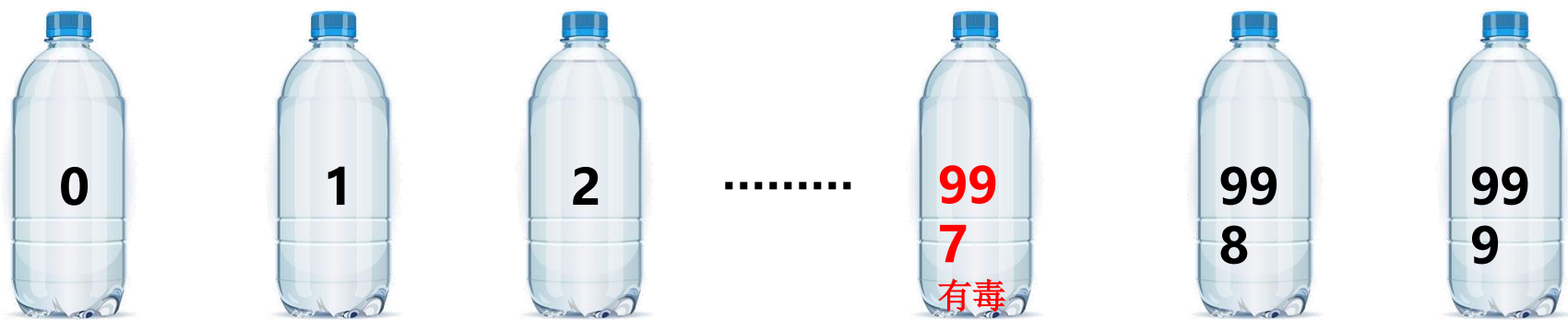


0000000000 0000000001 0000000010 **1111100101** 1111100110 1111100111



一个示例

【求解】小白鼠验毒问题之求解



0000000000 0000000001 0000000010 1111100101 1111100110

1111100111

0 1位
1 $2^1=2$ 个

00
01 2位
10 $2^2=4$ 个
11

000
001
010
011 3位
100 $2^3=8$ 个
101
110
111

9位
 $2^9=512$ 个

10位
 $2^{10}=1024$ 个



示例背后的思维

0 和 1

多种不同的含义均用**01**串表达

- 不同的两种状态都可表达为**0**和**1**

---小白鼠- “死” 与 “活” ；

---小白鼠对某瓶水- “喝” 与 “不喝” ；

---水 “有毒” 与 “无毒”

- 用**0/1**编码串表示不同的含义

---**000010**---对应第(**000010**的十进制)瓶水

---**000010**---第*i*位对应第*i*只小白鼠，**1**喝**0**不喝；

---**000010**---第*i*位对应第*i*只小白鼠，死**1** 活**0** ；

00001

多种不同的含义统一在一个**01**串中，可无缝隙实现含义转换

计算思维
理解的
关键点

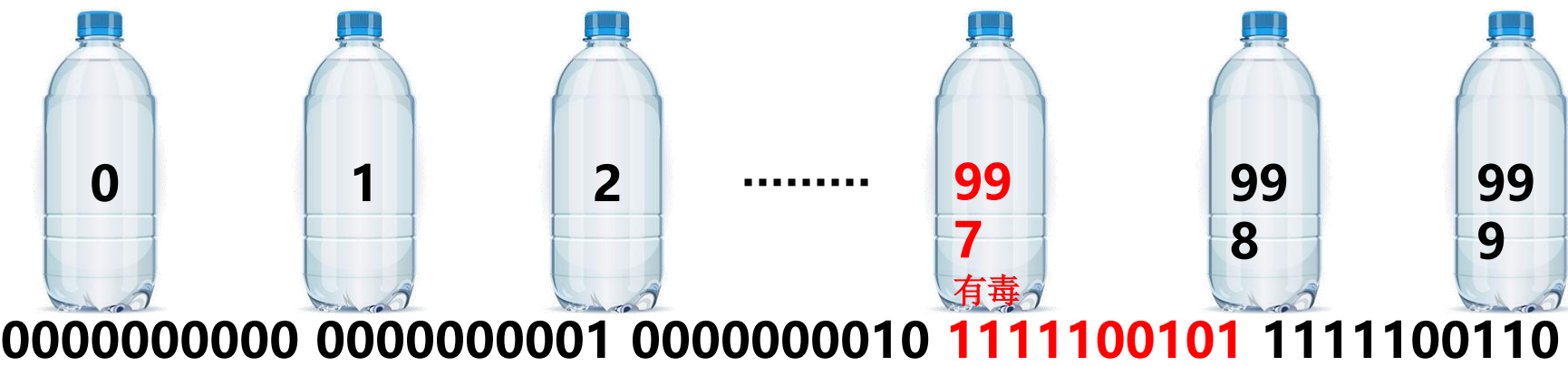
难度：【**0**和**1**的多种语义的组合应用】

小白鼠 “喝” 与 “不喝” 与 小白鼠 “死” 与 “活” ，
水的 “有毒” 与 “无毒”

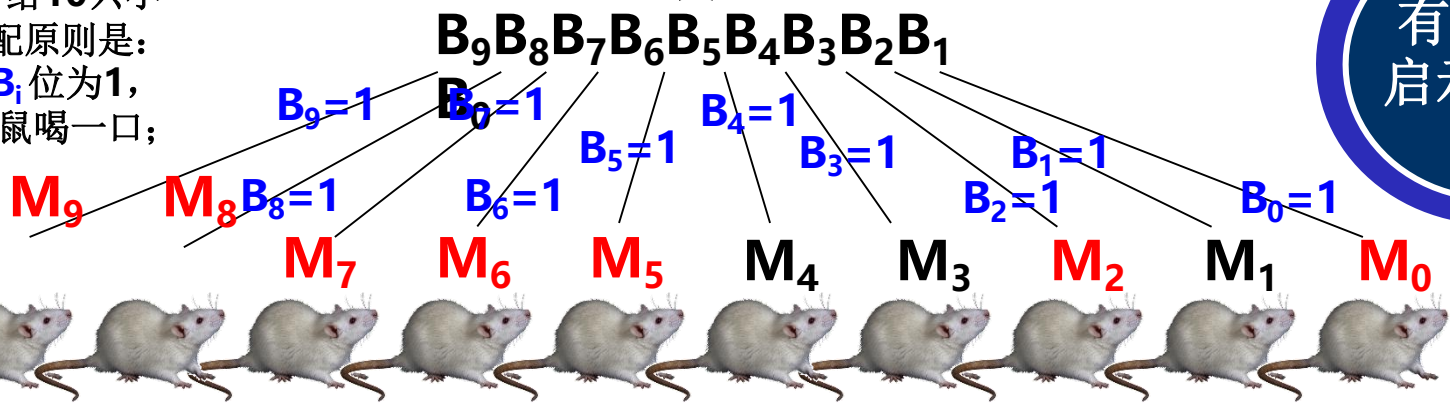


一个示例

【求解】小白鼠验毒问题之求解



1000瓶水分给10只小白鼠喝，分配原则是：如一瓶水的 B_i 位为1，则给 M_i 小白鼠喝一口；



有什么启示吗？

哪些小白鼠死掉了？死掉为1，不死为0： $M_9M_8M_7M_6M_5M_4M_3M_2M_1M_0 =$

1111100101



Bit-level operations



计算机是怎样完成计算的？

计算机器是怎样完成加法运算的，你知道吗？

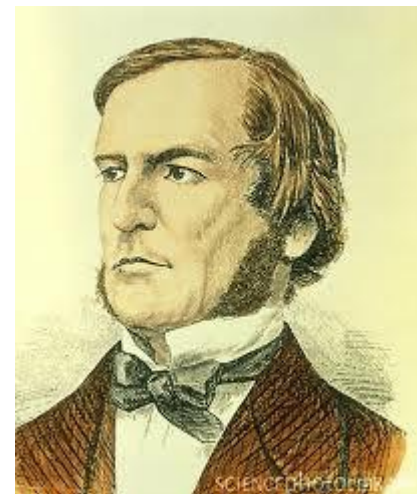
1+1=2
2+1=3 2+2=4
3+1=4 3+2=5 3+3=6
4+1=5 4+2=6 4+3=7 4+4=8
5+1=6 5+2=7 5+3=8 5+4=9 5+5=10
6+1=7 6+2=8 6+3=9 6+4=10 6+5=11 6+6=12
7+1=8 7+2=9 7+3=10 7+4=11 7+5=12 7+6=13 7+7=14
8+1=9 8+2=10 8+3=11 8+4=12 8+5=13 8+6=14 8+7=15 8+8=16
9+1=10 9+2=11 9+3=12 9+4=13 9+5=14 9+6=15 9+7=16 9+8=17 9+9=18

0+0 = 0
0+1 = 1
1+0 = 1
1+1= 10



布尔代数

- George Boole(1815-1864)发明
 - 逻辑的代数表示:
 - Encode “True” as 1
 - Encode “False” as 0
- Claude Shannon(1916–2001)建立了信息论
 - 建立布尔代数和数字逻辑之间的关联
- 在数字电路设计和分析中起最重要的作用





计算机的本质是逻辑

【逻辑】与基本逻辑运算

逻辑是指事物因果之间所遵循的规律，是现实中普适的思维方式

- ◆ 逻辑的基本表现形式是**命题**与**推理**，
- ◆ 推理即依据由简单命题的判断推导得出复杂命题的判断结论的过程。

【命题】内容为“真”或为“假”的一个判断语句！

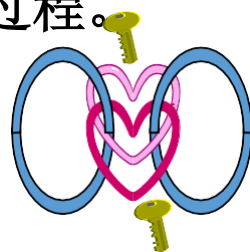
【命题的运算】

【与】运算(**AND**): 当**X**和**Y**都为真时, **X AND Y**也为真; 否则假。

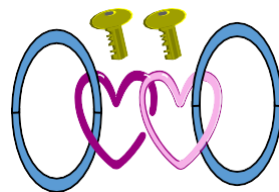
【或】运算(**OR**): 当**X**和**Y**都为假时, **X OR Y**也为假; 否则为真。

【非】运算(**NOT**): 当**X**为真时, **NOT X**为假; 当**X**为假时, **NOT X**为真。

【异或】运算(**XOR**): 当**X**和**Y**都为真或都为假时, **X XOR Y**为假;
否则, **X XOR Y**为真。



“与”运算：两把钥匙都有才能开门



“或”运算：只要有任何一把钥匙便能开门



布尔代数

0和1的计算：基本逻辑运算

注: 1表示 真, 0表示 假

用0和1来表示逻辑运算

- **【AND】**：“与”运算
有0为0，全1为1
- **【OR】**：“或”运算
有1为1，全0为0
- **【NOT】**：“非”运算
非0则1，非1则0
- **【XOR】**：“异或”运算
相同为0，不同为1

0	0	1	1
AND 0	AND 1	AND 0	AND 1
0	0	0	1
0	0	1	1
OR 0	OR 1	OR 0	OR 1
0	1	1	1
NOT 0		NOT 1	
1		0	
0	0	1	1
XOR 0	XOR 1	XOR 0	XOR 1
0	1	1	0



布尔代数

- 位向量运算
- 位粒度的操作

$$\begin{array}{r} 01101001 \\ \& 01010101 \\ \hline 01000001 \end{array}$$

$$\begin{array}{r} 01101001 \\ | 01010101 \\ \hline 01111101 \end{array}$$

$$\begin{array}{r} 01101001 \\ ^ 01010101 \\ \hline 00111100 \end{array}$$

$$\begin{array}{r} \sim 01010101 \\ \hline 10101010 \end{array}$$



布尔代数

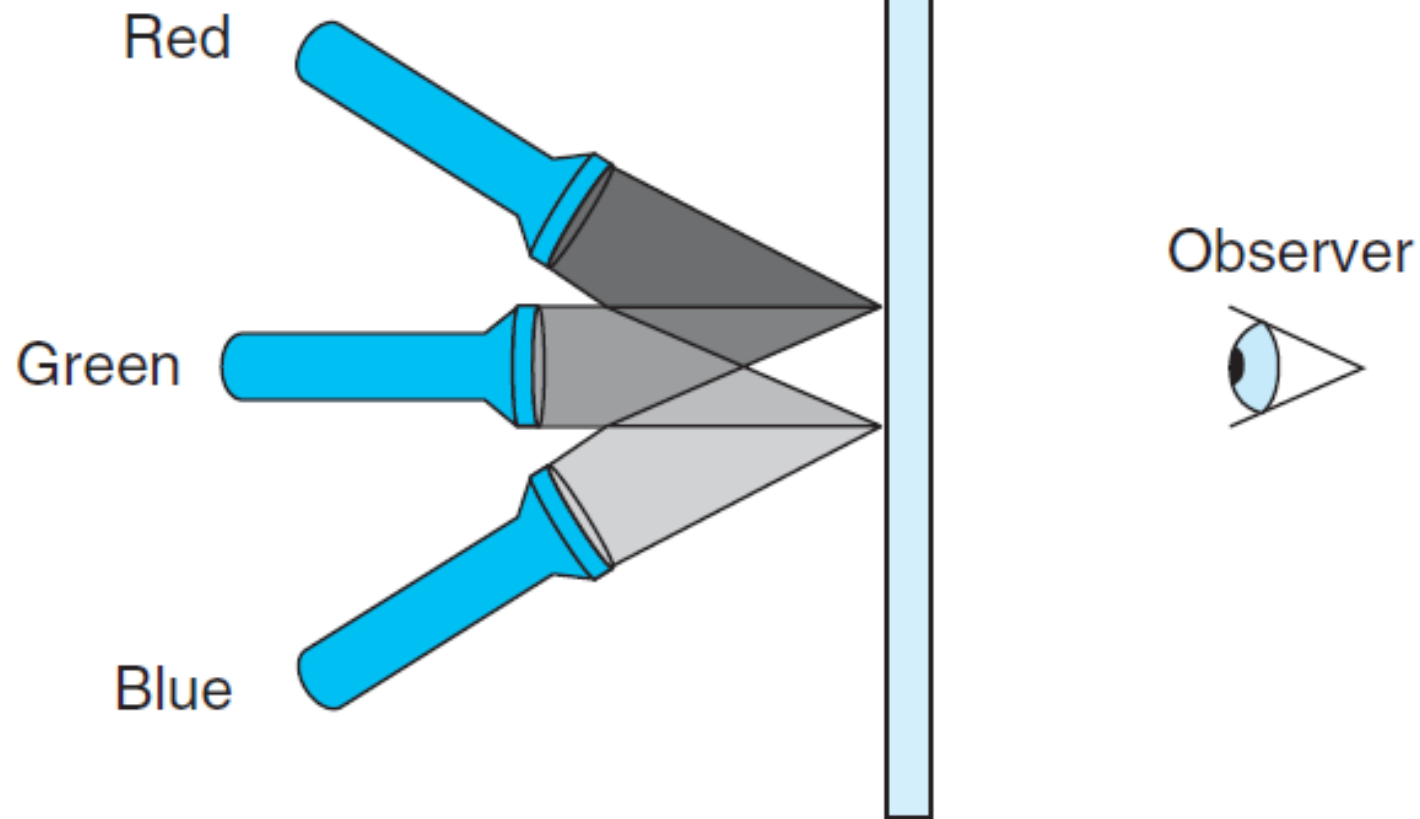
- 集合的形式表达
- 用宽度为 w bit的向量表示子集 $\{0, \dots, w-1\}$
 - $a_j = 1$ if $j \in A$
 - 01101001 $\{0, 3, 5, 6\}$
 - 01010101 $\{0, 2, 4, 6\}$
 - & Intersection 01000001 $\{0, 6\}$
 - | Union 01111101 $\{0, 2, 3, 4, 5, 6\}$
 - ~ Complement 10101010 $\{1, 3, 5, 7\}$
 - ^ Symmetric difference 00111100 $\{2, 3, 4, 5\}$



RGB Color Model

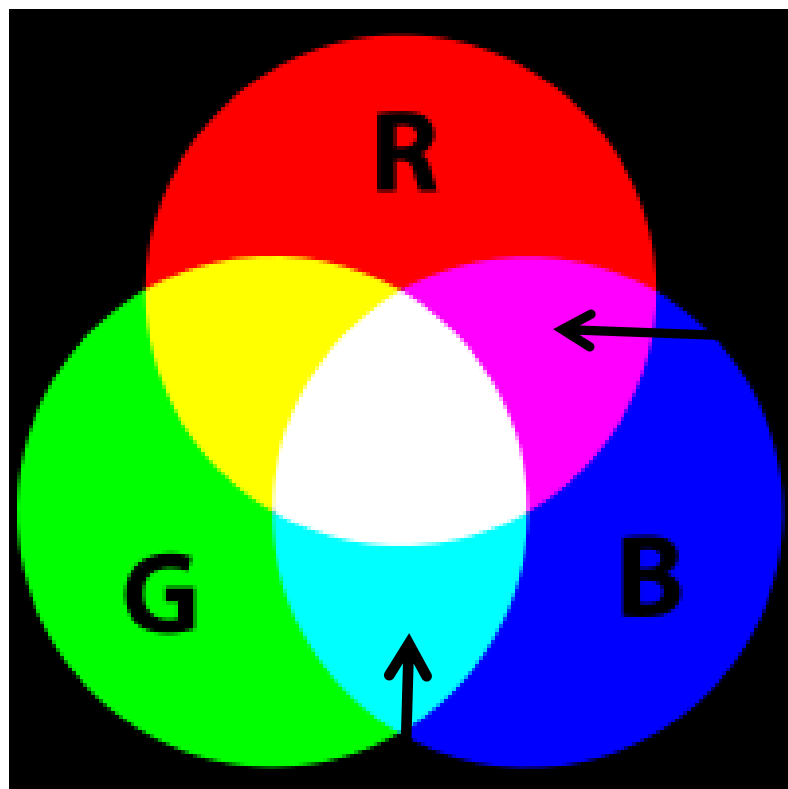
Light sources

Glass screen





RGB Color Model



Magenta

Cyan

<i>R</i>	<i>G</i>	<i>B</i>	Color
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White



RGB Color Model

- 上面8种颜色各自的补色是什么？
- 一种颜色的补色是指：
 - 关掉刚才亮的灯
 - 打开刚才灭的灯

<i>R</i>	<i>G</i>	<i>B</i>	Color
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White

- 描述在下面颜色上应用以下布尔运算带来的效果：

Blue | Green =

Yellow & Cyan =

Red ^ Magenta =

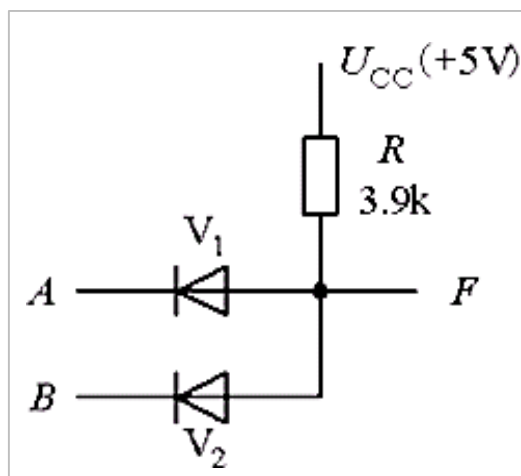


用电子技术实现基本逻辑运算

基本逻辑运算的电子实现：逻辑门

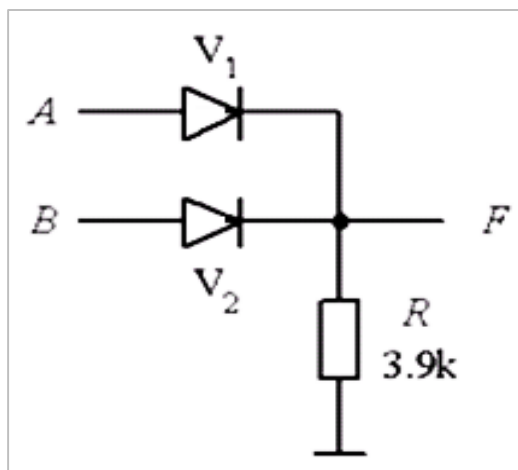
电信号：**0V**（低电平） 和 **5V**（高电平）

用二极管、三极管实现基本逻辑运算的电路



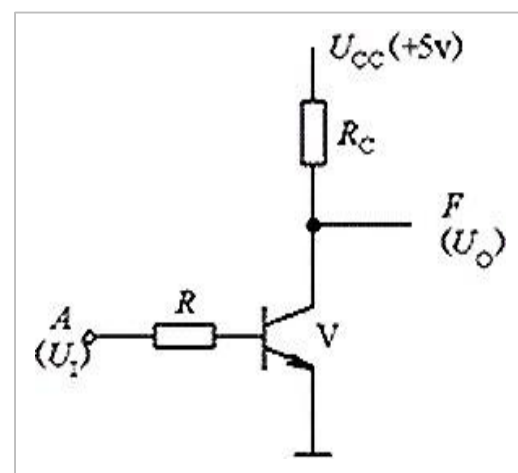
$$F = A \text{ AND } B$$

【与门】电路



$$F = A \text{ OR } B$$

【或门】电路



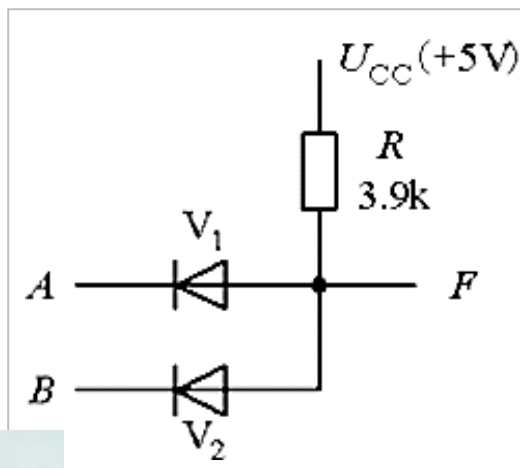
$$F = \text{NOT } A$$

【非门】电路

这些电路被封装成集成电路(芯片)，即所谓的逻辑门：【与门】、【或门】和【非门】。



集成电路示意

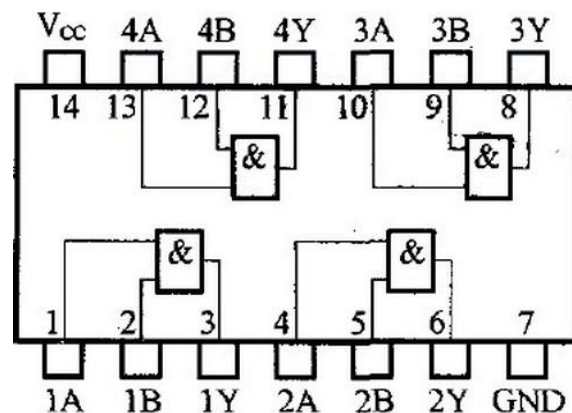


$$F = A \text{ AND } B$$

【与门】电路



【与门】电路符号
左侧是输入，右侧是输出

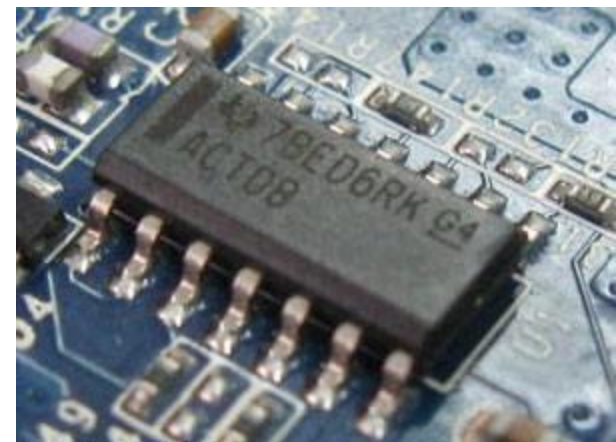
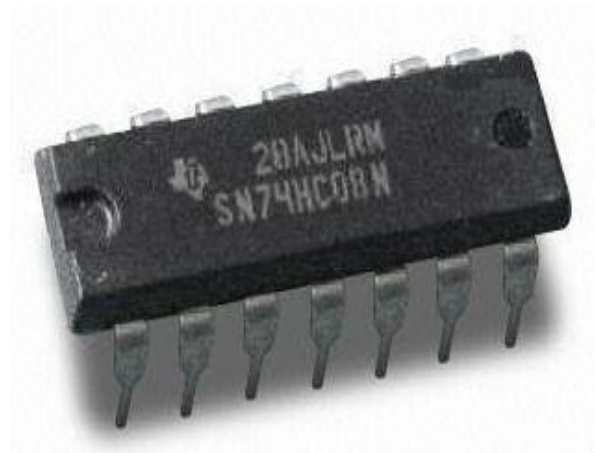


1Y = 1A AND 1B

2Y = 2A AND 2B

3Y = 3A AND 3B

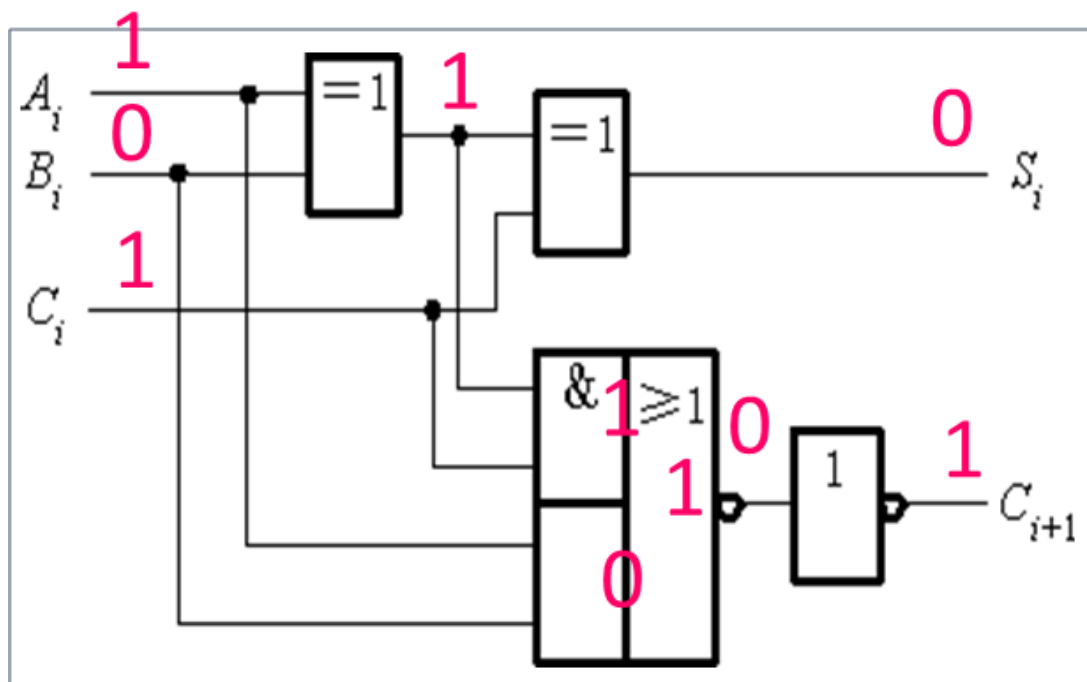
4Y = 4A AND 4B





加法运算是这样实现的

一位加法器的示例

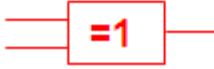


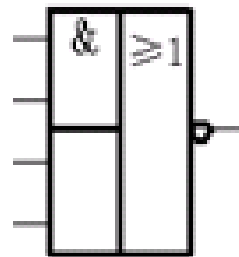
$$\begin{array}{r} A_i \\ B_i \\ C_i \\ + \\ \hline C_{i+1} \quad S_i \end{array}$$

 &
与门电路符号

 ≥ 1
或门电路符号

 1
非门电路符号

 =1
异或门电路符号





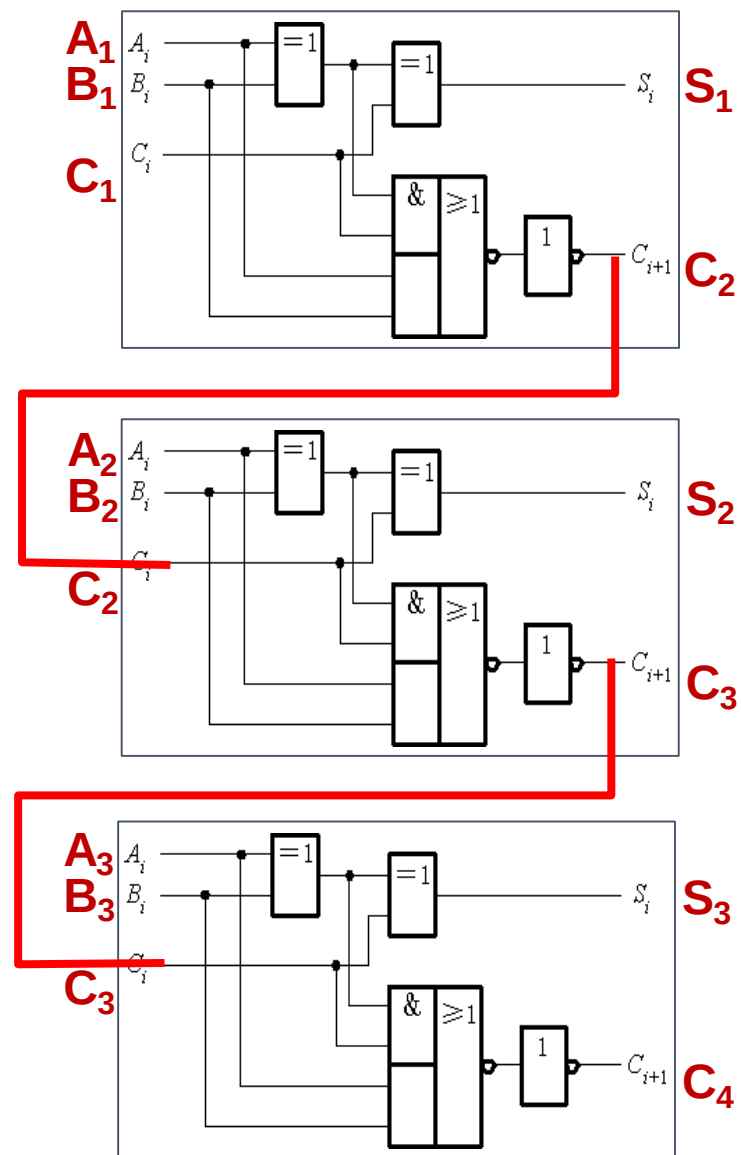
加法运算是这样实现的

多位加法器的实现

【分层构造】低层电路已验证正确，可被封装起来；用已封装的已验证的低层电路可构造更为复杂的高层电路；如此一层层构造。

$$\begin{array}{r} \\ + \\ \hline \end{array} \begin{array}{r} A_3 \\ B_3 \\ \hline C_4 \\ S_3 \end{array}$$

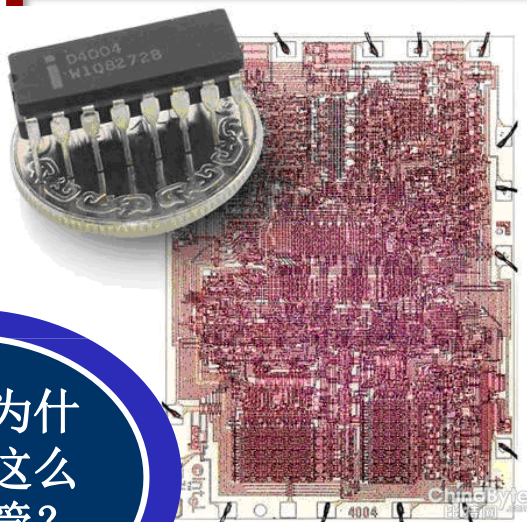
$$\begin{array}{r} \\ + \\ \hline \end{array} \begin{array}{r} 0 \\ 0 \\ \hline 1 \\ 1 \end{array}$$





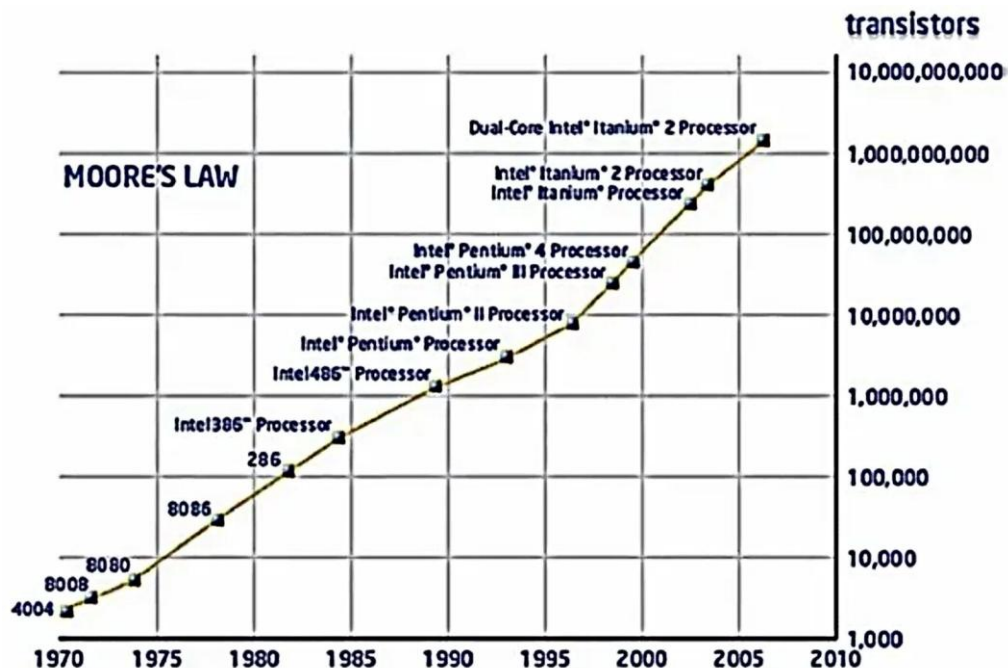
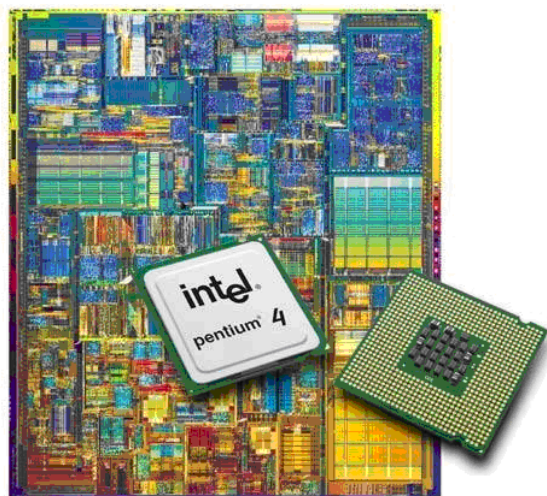
集成电路是这样复杂起来的

集成电路是这样复杂起来的...



疑问：为什么需要这么多晶体管？

摩尔定律：芯片可容纳的晶体管数目大约每经过**18**个月便会增加一倍：**2021**年达到的规模一块芯片**4.6**万亿颗晶体管



从沙子到芯片-中芯国际



Bit-Level Operations in C

- Operations $\&$, $|$, $\sim$, $\wedge$ Available in C
 - 可以应用于任何整数数据类型
 - long, int, short, char
 - 将参数视为bit vectors
 - 参数中每一位对应去做位运算（并行）



Bit-Level Operations in C

- Examples (Char data type)

- $\sim 0x41 \ \& \ 0xBE$

- $\sim 01000001_2 \ \& \ 10111110_2$

- $\sim 0x00 \ \& \ 0xFF$

- $\sim 00000000_2 \ \& \ 11111111_2$

- $0x69 \ \& \ 0x55 \ \& \ 0x41$

- $01101001_2 \ \& \ 01010101_2 \ \& \ 01000001_2$

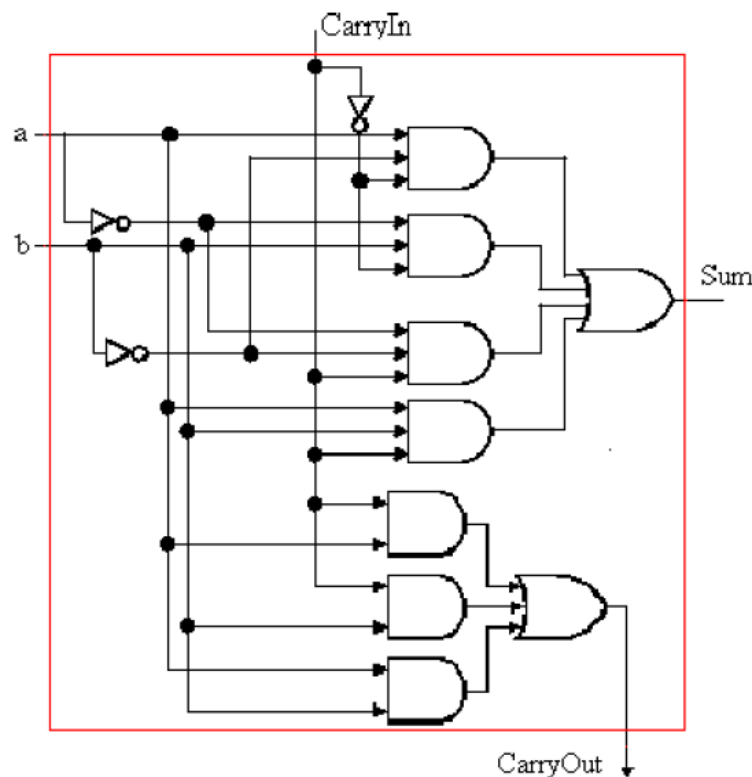
- $0x69 \ | \ 0x55 \ | \ 0x7D$

- $01101001_2 \ | \ 01010101_2 \ | \ 01111101_2$



Cool Stuff with Xor

- 如何比较两个数是否相等？
- Bitwise Xor 可以用来做加法
 - $0+0=0$
 - $1+1=0$ (有额外进位)
 - $1+0=0+1=1$
- 每个bit都是自己的加法逆元
 - 相加本位得0
 - $A \wedge A = 0$
- Xor用于比较两个bit group是否相等
 - 每个bit都相等，则得到0
 - 有一个bit不等，则得到非0





Cool Stuff with Xor

```
int inplace_swap(int *x, int *y)
{
    *x = *x ^ *y;  /* #1 */
    *y = *x ^ *y;  /* #2 */
    *x = *x ^ *y;  /* #3 */
}
```

Step	*x	*y
Begin	A	B
1	$A \oplus B$	B
2	$A \oplus B$	$(A \oplus B) \oplus B = A \oplus (B \oplus B) = A \oplus 0 = A$
3	$(A \oplus B) \oplus A = (B \oplus A) \oplus A = B \oplus (A \oplus A) = B \oplus 0 = B$	A
End	B	A

$$\begin{aligned} A \oplus 0 &= A \\ A \oplus 1 &= \sim A \end{aligned}$$



Cool Stuff with Xor

```
int inplace_swap(int *x, int *y)
{
    *x = *x ^ *y;  /* #1 */
    *y = *x ^ *y;  /* #2 */
    *x = *x ^ *y;  /* #3 */
}
```

```
1 void reverse_array(int a[], int cnt) {
2     int first, last;
3     for (first = 0, last = cnt-1;
4         first <= last;
5         first++, last--)
6         inplace_swap(&a[first], &a[last]);
7 }
```



掩码 (Mask) 操作

- Bit pattern
 - 0xFF
 - Having 1s for the least significant eight bits
 - Indicates the lower-order byte of a word
- Mask Operation
 - $X = 0x89ABCDEF$
 - $X \& 0xFF = ?$
- 实现？ ？



Mask Operations

- Write C expressions that work for any word size $w \geq 8$
- For $x = 0x87654321$, with $w = 32$
- The least significant byte of x , with all other bits set to 0
 - `[0x00000021]`.



Mask Operations

- Bit pattern
 - 0xFF
- Mask Operation
 - $X = 0x89ABCDEF$
 - $X \mid 0xFF = ?$
- 实现？？
-



Mask Operations

- Bit pattern
 - 0xFF
- Mask Operation
 - $X = 0x89ABCDEF$
 - $X \wedge 0xFF = ?$
- 实现？？

取反



Mask Operations

- 掩码操作功能总结
 - 与操作
 - 保留某些位，其他位设为0
 - 或操作
 - 某些位强制置1
 - 异或操作
 - 某些位取反 (mask的对应位为1时)



课堂练习: Bis & Bic

- DEC公司的VAX计算机: 没有And和Or指令, 只有bis和bic指令: Set result z to x and modify it
- $z = \text{bis}(\text{int } x, \text{int } m)$ (bit set)
 - Set result z to 1 at each bit position where m is 1
- $z = \text{bic}(\text{int } x, \text{int } m)$ (bit clear)
 - set result z to 0 at each bit position where m is 1
- Use bis and bic to implement
 - $\text{Or}(\text{int } x, \text{int } y)$
 - $\text{Xor}(\text{int } x, \text{int } y)$

X	Y	Output
0	0	0
0	1	1
1	0	1
1	1	0

Void Or (int x, int y) { bis(x, y); }

Bic(x, y)相当于And(x, -y)

$A \wedge B = A(\sim B) + (\sim A)B$

Void Xor (int x, int y) { bis(bic(x,y), bic(y,x)); }



Logical Operations in C

- Bit level Operations $\&$, $|$, $\sim$, $\wedge$
 - 置1, 清零0, 取反
- Logical Operators
 - $\&\&$, $||$, $!$
 - 将0视为“False”
 - 任何非0值都被视为“True”
 - 总是return 0 or 1
 - 提前终止 (短路表达式)



Logical Operations in C

- Examples (char data type)
 - `!0x41` $\rightarrow$ `0x00`
 - `!0x00` $\rightarrow$ `0x01`
 - `!!0x41` $\rightarrow$ `0x01`
 - `0x69 & 0x55` $\rightarrow$ `0x01`
 - `0x69 | 0x55` $\rightarrow$ `0x01`



Short Cut in Logical Operations

- `a && 5/a`
 - If `a` is zero, the evaluation of `5/a` is stopped
 - avoid division by zero
- 练习: Using only bit-level and logical operations
 - Implement `x == y`
 - it returns 1 when `x` and `y` are equal, and 0 otherwise

! $(x \wedge y)$



Shift Operations in C

- Left Shift: $x \ll y$
 - 将bit-vector x 左移 y 位
 - 左边多出来的bits丢掉
 - 右边补0

Argument x	01100010
$\ll 3$	00010000

Argument x	10100010
$\ll 3$	00010000



Shift Operations in C

- Right Shift: $x \gg y$
 - 将bit-vector x 向右移动 y 位
 - 丢掉右边额外的bits
 - 逻辑移位（无符号数）
 - 左边补0
 - 算数移位（有符号数）
 - 左边总是补入最左边的符号位
 - 在补码表示时很有用（移动后保持数字符号不变，正数 $\rightarrow$ 正数，负数 $\rightarrow$ 负数）
 - 未定义的行为
 - Shift amount < 0 or $\geq$ word size

Argument x	01100010
Log. $\gg 2$	00011000
Arith. $\gg 2$	00011000

Argument x	10100010
Log. $\gg 2$	00101000
Arith. $\gg 2$	11101000



Shift Operations in C

- What happens ?
 - `int lval = 0xFEDCBA98 << 32;`
 - `int aval = 0xFEDCBA98 >> 36;`
 - `unsigned uval = 0xFEDCBA98u >> 40;`
- It may be
 - `lval` `0xFEDCBA98` (0)
 - `aval` `0xFFEDCBA9` (4)
 - `uval` `0x00FEDCBA` (8)
- Be careful about
 - `1<<2 + 3<<4` means `1<<(2 + 3)<<4`



课堂练习

- 写出代码实现如下函数：
 - /* Return 1 when x contains an **even** number of 1s; 0 otherwise.
 - Assume w=32 */
 - int even_ones (unsigned x);
 - 你的代码最多只能包括12个算术运算、位运算和逻辑运算。
 - C语言中的位运算：&, |, ~, ^ (与、或、非、异或)；移位运算<<, >>



课堂练习（答案）

```
int even_ones (unsigned x) {  
    x=x^(x>>16);  
    x=x^(x>>8);  
    x=x^(x>>4);  
    x=x^(x>>2);  
    x=x^(x>>1);  
    return !(x&1);  
}
```



课堂练习

写出代码实现以下函数：

```
/* * Generate mask indicating leftmost 1 in x. Assume w=32
```

```
* For example, 0xFF00 -> 0x8000, and 0x6000 -> 0x4000.
```

```
* If x = 0, then return 0 */
```

```
int leftmost_one(unsigned x);
```

可以假设x是32位的int类型，代码最多包含15个算数运算、位运算或逻辑运算。

提示：现将x转为[0...01...1]的位向量



```

0 1 x x x x x x x x x x x x x x x x x x x x x x x x x x x x
| 0 1 x x x x x x x x x x x x x x x x x x x x x x x x x x x x
-----
0 1 1 x x x x x x x x x x x x x x x x x x x x x x x x x x x x

```

```
x = x | x>>2;
```

```

0 1 1 x x x x x x x x x x x x x x x x x x x x x x x x x x
| 0 1 1 x x x x x x x x x x x x x x x x x x x x x x x x x x
-----
0 1 1 1 1 x x x x x x x x x x x x x x x x x x x x x x x x

```

$$X = X \mid X > 4;$$
[illegible]



```
/*
 * Generate mask indicating leftmost 1 in x. Assume w=32
 * For example, 0xFF00 -> 0x8000, and 0x6000 -> 0x4000.
 * If x = 0, then return 0
 */
int leftmost_one(unsigned x) {
    /*
     * first, generate a mask that all bits after leftmost one are one
     * e.g. 0xFF00 -> 0xFFFF, and 0x6000 -> 0x7FFF
     * If x = 0, get 0
     */
    x |= x >> 1;
    x |= x >> 2;
    x |= x >> 4;
    x |= x >> 8;
    x |= x >> 16;
    /*
     * then, do mask & (~mask >> 1), reserve leftmost bit one
     * that's we want
     */
    return x & (~x >> 1);
}
```



课堂练习: bitCount

- Returns number of 1's a in word
- Examples: `bitCount(5) = 2`, `bitCount(7) = 3`
- Legal ops: `! ~ & ^ | + << >>`
- Max ops: 40

x & mask;

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

& 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1

0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0

x >>1 & mask;

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

& 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1

0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0

x >>2 & mask;

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

& 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1

0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 0

x >>3 & mask;

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

& 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1

0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 1

x & mask;

1	1	0	0	0	1	0	0	1	0	1	1	0	1	1	1	1	0	1	0	0	0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

&

0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

x >>1 & mask;

1	1	0	0	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	0	0	0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

&

0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

x >>2 & mask;

1	1	0	0	0	1	0	0	1	0	1	1	1	0	1	1	1	0	1	0	0	0	1	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

&

0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	1	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

x >>3 & mask;

1	1	0	0	0	1	0	0	1	0	1	1	1	1	0	1	0	0	0	1	1	1	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

&

0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

```
x & mask;
```

000000000000100010001000000010000

```
x >>1 & mask;
```

00000001000000010001000000010000

```
x >>2 & mask;
```

00010000000100000001000100010000

```
x >>3 & mask;
```

00010000000000001000100000000001

S+=x & mask;

S+=x >>1& mask;

S+=x >>2& mask;

S+x >>3& mask;

	1	1	0	0	0	0	1	0	0	1	0	1	1	1	1	1	0	1	0	0	0	1	1	1	1	0	0	0		
	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0		
	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0		
	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0		
	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1		
&	0	0	1	0	0	0	0	1	0	0	1	0	0	1	1	0	1	0	0	0	0	1	0	0	1	1	0	0	0	1

00100001001000110100000100110001

00100001001000110100000100110001

`s = s + (s >> 16);`

+

`mask = 0xF | (0xF << 8);`

`s = (s & mask) + ((s >> 4) & mask);`

0110001001010100

+

0000100000001001

+

`(s + (s >> 8)) & 0x3F`

10001

17

80



Sum 8 groups of 4 bits each

```
int bitCount(int x) {  
    int m1 = 0x11 | (0x11 << 8);  
    int mask = m1 | (m1 << 16);  
    int s = x & mask;  
    s += x>>1 & mask;  
    s += x>>2 & mask;  
    s += x>>3 & mask;  
    s = s + (s >> 16);  
    mask = 0xF | (0xF << 8);  
    s = (s & mask) + ((s >> 4) & mask);  
    return (s + (s>>8)) & 0x3F;  
}
```



```
int m1 = 0x11 | (0x11 << 8);  
int mask = m1 | (m1 << 16);  
s = x & mask;
```

Mask:

00010001000100010001000100010001

11000010010110111111010001111000
00010001000100010001000100010001

0000000000001000100010000000010000



方法二

// Create masks

```
M5 = ~((-1) << 32); // 032132
M4 = M5 ^ (M5 << 16); // (016116)2
M3 = M4 ^ (M4 << 8); // (0818)4
M2 = M3 ^ (M3 << 4); // (0414)8
M1 = M2 ^ (M2 << 2); // (0212)16
M0 = M1 ^ (M1 << 1); // (01)32
```

// Compute popcount

```
x = ((x >> 1) & M0) + (x & M0);
x = ((x >> 2) & M1) + (x & M1);
x = ((x >> 4) + x) & M2;
x = ((x >> 8) + x) & M3;
x = ((x >> 16) + x) & M4;
x = ((x >> 32) + x) & M5;
```

Notation:

$X^k = \underbrace{XX \cdots X}_{k \text{ times}}$

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

+

1 0 0 0 0 0 0 1 0 1 0 1 0 1 1 0 1 0 1 0 0 1 0 0 0 1 1 0 0 1 0 0

X&M0

(X>>1)&M0

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

+ 1 0 0 1 0 0 1 1 1 1 0 0 0 1 1 0

(X>>1)&M0

1 0 0 0 0 0 1 0 1 0 1 0 1 1 0 1 0 1 0 0 1 0 0 0 1 1 0 0 1 0 0

X&M1

(X>>2)&M1

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

(X>>>1)&M0

+ 1 0 0 1 0 0 1 1 1 1 0 0 0 1 1 0

00 01 01 10 10 00 10 00

X&M1

(X>>>2)&M1

+ 10 00 01 01 10 01 01 01

001000010010001101000000100110001

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

+

1 0 0 1 0 0 1 1 1 1 0 0 0 0 1 1 0

(X>>>1)&M0

0 0 0 1 0 1 1 0 1 0 0 0 1 0 0 0

X&M1

+

1 0 0 0 0 1 0 1 1 0 0 1 0 1 0 1

(X>>>2)&M1

0 0 1 0 0 0 0 1 0 0 1 0 0 0 1 1 0 1 0 0 0 0 1 0 0 1 1 0 0 0 1

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

+

1 0 0 1 0 0 1 1 1 1 0 0 0 1 1 0

(X>>1)&M0

00 01 01 10 10 00 10 00

X&M1

+

10 00 01 01 10 01 01 01

(X>>2)&M1

0001

0011

0001

0001

X&M2

+

0010

0010

0100

0011

(X>>4)&M2

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1 0 0

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 0 0 1 1 1 1 0 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

+

1 0 0 1 0 0 1 1 1 1 0 0 0 0 1 1 0

(X>>1)&M0

00 01 01 10 10 00 10 00

X&M1

+

10 00 01 01 10 01 01 01

(X>>2)&M1

0001 0011 0001 0001

X&M2

+

0010 0010 0100 0011

(X>>4)&M2

0 1 0 1 1 0 1 1

0 0 0 0 0 1 0 0

X&M3

1 1 0 0 0 0 1 0

0 0 0 0 0 1 0 1

(X>>8)&M3

+

0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 0 0 1 1 1 1 0 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

+

1 0 0 1 0 0 1 1 1 1 0 0 0 0 1 1 0

(X>>1)&M0

00 01 01 10 10 00 10 00

X&M1

+

10 00 01 01 10 01 01 01

(X>>2)&M1

0001 0011 0001 0001

X&M2

+

0010 0010 0100 0011

(X>>4)&M2

0 1 0 1 1 0 1 1

0 0 0 0 0 1 0 0

X&M3

1 1 0 0 0 0 1 0

0 0 0 0 0 1 0 1

(X>>8)&M3

+

0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1

X&M4

0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0

(X>>16)&M4

1 1 0 0 0 0 1 0 0 1 0 1 1 0 1 1 1 1 1 1 0 1 0 0 0 1 1 1 1 0 0 0

1 0 0 0 1 1 0 1 1 1 1 1 0 1 1 0 0

X&M0

+ 1 0 0 1 0 0 1 1 1 1 0 0 0 1 1 0

(X>>1)&M0

00 01 01 10 10 00 10 00

X&M1

+ 10 00 01 01 10 01 01 01

(X>>2)&M1

0001 0011 0001 0001

X&M2

+ 0010 0010 0100 0011

(X>>4)&M2

0 1 0 1 1 0 1 1 0 0 0 0 0 1 0 0

X&M3

1 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1

(X>>8)&M3

+

0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1

X&M4

+ 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0

(X>>16)&M4

0 1 0 0 0 1



17



双节大礼包

- 作业

- Homework1提交
- 时间： 10月8日23:59
- 形式： pdf文件， 上传obe.ruc.edu.cn

- 实验

- 内容： Lab1 DataLab